

SFU

SIMON FRASER
UNIVERSITY



ENSC 452

Geometry Dash FPGA

G05: Artin, Ali

Project Overview



Core Objective

Replicate the mechanics of Geometry Dash (auto-scrolling, rhythm-based jumping) entirely on the Zynq SoC



Architecture

Utilizes a bare-metal Dual-Core approach, splitting game logic/video and audio processing across two ARM processors.



Hardware/Software Integration

Combines software-driven state machines with custom FPGA hardware blocks (IPs) for VGA rendering, LFSR random number generation, and audio threshold detection.

Project Milestones & Goals

#	Lead	Key Milestone Deliverable
1	Ali & Artin	Dual-core system integration for audio/video; initial VGA sprite rendering and keyboard interaction.
2	Ali & Artin	Multi-menu navigation (gameplay, pause, menus) and implementation of button-triggered 'jump' mechanics.
3	Artin	Auto-scrolling map with hardware-based audio threshold detection for synchronized obstacle generation.
4	Ali	LFSR-based pseudo-random obstacle generation; full collision detection and music-synced gameplay.

Base Goals

Multi-mode gameplay, collision detection, VGA output, and audio playback.

Reach Goals

Dynamic pause menu, level selector, gamemodes.

Challenges & Changes

Challenges



Hardware Integration

Combining the VGA controller and Audio codec hardware IPs.



Complex Synchronization

Keeping the VGA rendering, hardware audio detection, and SD stream synced.

Changes



SD Card Audio Loading

Shifted to full SD integration for dynamic loading of WAV files.



Player Customization

Added sprite selection state for users to choose between multiple player icons.

Results & Outcomes



Fully Playable 60 FPS Game

A responsive, bare-metal rhythm platformer running smoothly with physics and accurate collision detection.



Three Distinct Game Modes

Features a hardcoded *Level 1*, an *Endless Mode* (hardware LFSR), and an *Audio Mode* driven by threshold detection.



Math-Based Graphical Menu

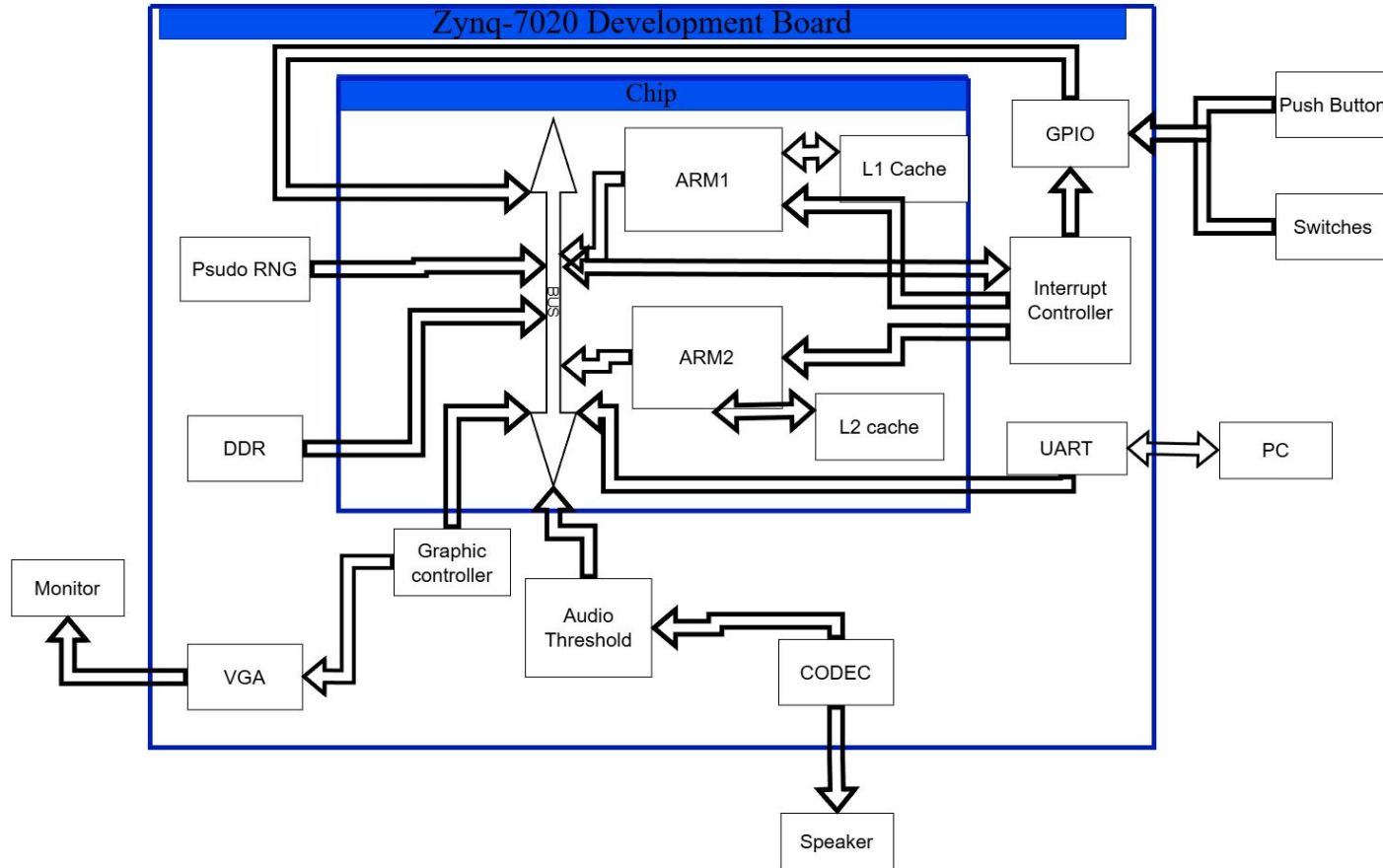
Replaced static .data images with math-generated C++ graphics. 8x8 font open source library for text.



State Management

Switching between gameplay, sprites, and menus with zero memory leaks and high-score tracking.

System Architecture (High-Level)



Hardware Blocks: LFSR & Audio Detector



1. LFSR

- 8-bit LFSR to extend gameplay before patterns repeat
- Programmable threshold register to control spike generation rate
- Seed and reset inputs for reproducible or re-randomized obstacle sequences



2. Audio Threshold Detector

- Threshold to tune sensitivity to a specific audio track
- Accepts audio input from live stream or on-board memory
- Generates spike trigger signals from the audio waveform for obstacle creation

Software & Dual-Core Orchestration

Core 0 (Master Game Engine)

main.cc: Game physics, level generator, interrupts, VGA rendering, state machines, handling 60 FPS frame limiter, and SD card initialization.



Core 1 (Audio Engine)

adventures_with_ip.c: Dedicated entirely to configuring ADAU1761 audio codec and streaming I2S audio without interruption.

Shared Memory Mailbox

Core 0 controls Core 1 using a shared pointer (**0xFFFF0000**) as a state flag: 0=Pause, 1=Play, 2=Reset.

Cache Management

Core 1 disables data cache (**Xil_DCacheDisable**) for instant mailbox reads from Core 0 without stale data.

Design Process & Methodology

Hybrid Approach: Combined bottom-up design with top-down development.

1. Hardware First (Bottom-Up)

Developed and integrated low-level IP blocks (VGA & audio) in Vivado.

2. Software Second (Top-Down)

Built the C++ game engine and logic before integrating hardware inputs.

Targeted Division of Work

Ali: VGA hardware & rendering, LFSR, UI graphics, and game state logic.

Artin: Audio integration & SD card, audio threshold detector, physics, synchronization.

Key Step: Early integration of individual hardware modules.

Conclusion & Lessons Learned

Conclusion

A Project to be Proud Of

Successfully built a complete, highly responsive rhythm platformer from scratch on a bare-metal architecture.

Major Degree Milestone

Completing ENSC 452 represents a significant achievement within Engineering Science.

Lessons Learned

Xilinx FPGA Dev Stack

Gained deep, hands-on experience navigating the complete pipeline between Vivado hardware synthesis and Vitis.

Advanced Hardware Design

Mastered the intricacies of custom IP creation, and AXI bus management.

Integration & Project Management

Learned how to effectively merge complex, independent subsystems while maintaining strict performance goals.

Demo / Q&A

Thank You!